CUDA

Programación en CUDA

Compute Unified Device Architecture (CUDA), es una plataforma de computación paralela y una Interfaz de Programación de Interfaces (API).

CUDA permite a los desarrolladores usar unidades de Nvida's graphic processing para procesamiento de propósito general.

- CUDA esta basado en el lenguaje C.
- El compilador de CUDA es NVCC.

Host vs Device

Tipicamente el Host se refiere al CPU y a la memoria global o RAM, mientras el Device se refire al GPU y a su memoria global. Tener claro esta distinción es crucial para poder construir nuestro software, ya que esto influye en la estructura del código, la transferencia de datos y en el manejo de memoria.

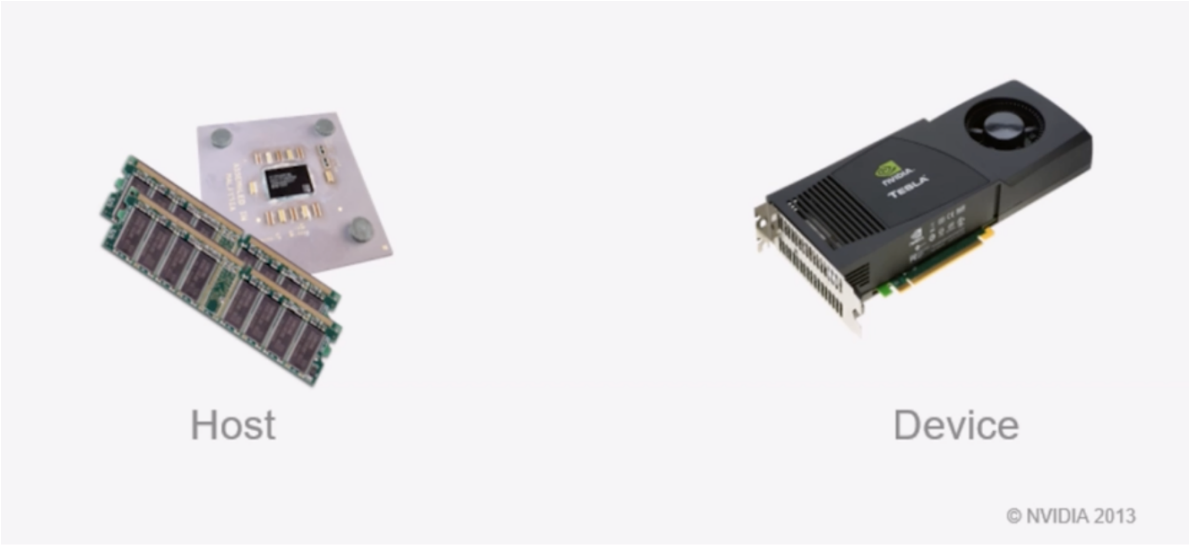

Figure 1: Host vs Device

Escribir un código en CUDA.

Es importante tener la noción de la jerarquía que existe a nivel de Hardware en una GPU. Consideremos.

- 1. GPU Completa
- 2. (SM) Streaming multiprocesor
- 3. Cada una de las particiones del streaming multiprocesor
- 4. Unidades computacionales de cada partición (FP32)

Esto es importante para tener la capacidad de mapear las distintas partes del código en sus respectivas jerarquias de software.

Por ello el primer paso para escribir una código de CUDA es **Dividir los GPU kernels de la aplicación** en pequeños segmentos de software, llamados **thread blocks**, por lo tanto el primer paso para configurar aplicaciones de CUDA es empezar configurando el número de bloques para cada **cuda kernel**. Esta partición corresponde a los dos primeras jerarquias del GPU, cada block se ejecutara en los diferentes setreaming multiprocesors.

Giga Thread Unit

Es una de las unidades más importantes dentro del GPU, una de sus funciones principales es asignar de forma eficiente estos bloques de thread (Block) a los distintos Streaming multiprocessors.

Asignar un block a un SM nos lleva a una siguiente pregunta, como asignar un solo bloque a las distintas particiones dentro el SM, ya que si utilizamos una sola partición para ejecutar el bloque completo estariamos desperdiciando recursos ya que las particiones restantes estarían en un estado de idle. Esta consideración nos lleva a tercer nivel de la jerarquía **warps**.

Cada bloque se puede dividir en pequeñas unidades conocidas como warps. Por ejemplo si deseamos dividir un bloque en 4 warps, entonces cada warp puede se asignado a cada unas de las particiones del SM. Pero ahora imaginemos que dividimos nuestros block en 32 warps y cada SM tiene 4 particiones entonces cada partición ejecutará 8 warps. En el caso que tenemos más tareas que unidades de hardware (8 warps para 1 partición), se requiere de un **manager**. Este por esta razón que tenemos una unidad llamada **warp scheduler**, en cada partición tenemos un warp scheduler ya que algunas asignamos muchos warps a una sola partición.

Ahora inspeccinomes el cuarto nivel dentro de la jerarquía, dentro de cada partición tenemos muchos cores, esto de lado del hardware, pero de lado del software un warp tipicamente consiste en 32 threads por omisión. Cada uno de estos hilos se ejecutará en cada uno de los cores individuales detro de la partición, de esta manera aseguramos que cada uno de los cores esta activo durante el proceso de cómputo. Enteder cada una de las 4 jerarquias es esencial para configurar de manera eficiente tu aplicación CUDA.

Para iniciar una aplicación CUDA, necesitamos especificar dos parámetros claves el número de bloques y el número de hilos dentro de cada bloque. Por lo que si queremos calcular el número de warps tenemos que dividir el numero de hilos de cada bloque entre 32, ya que es el número fijo de hilos que existen por cada warp es 32.

Hello World

```
#include <stdio.h>

// Kernerl: Function which is executed in the GPU
__global__ void test01() {
    //print the block and the threads IDs
    // in CUDA applications the CUDA numbers
    // can be distributed across different dimensions.
    printf("\nThe block ID is : %d", blockIdx.x);
    printf("\nThe thread ID is: %d", threadIdx.x);
}

int main() {
    test01 <<<1,1>>> ();
}
```

Pasos esenciales para una aplicación en CUDA

1. Asignar la memoria tanto en el host como en la GPU.
2. Copiar los valores del host en el device.
3. Definir el kernel
4. Ejecutar el kernerl con numero de hilos y bloques intercambiables.
5. Liberar la memoria del host y del device.

Occupancy

El cálculo teorico para la ocupación se obtene de la siguiente ecuación.

$$\frac{\text{warp used in a kernel}}{\text{max warps per SM}}$$

La ocupación teórica se refiere al máximo porcentaje de cómputo de la GPU que puede ser utilizado por el kernel bajo circunstancias ideales, asume las condiciones óptimas donde hay suficientes tareas independientes o suficientes hilos para ocupar completamente los recursos de la GPU sin producir cuellos de botella en la memoria.

Stall cycle

Ocurre cuando ninguno de los warps dentro de la partición esta listo para ejecutarse, es por ello que el valor de **archived occupancy** es menor que 100%. Esto puede ocurrir porque el registro de instrucciones posee instrucciones con dependencias que requieran que se finalice una intrucción para poder ejecutar otra instrucción.

Para conocer el valor de **archived occupancy** necesitamos conocer el total de ciclos activos y dividirlos entre el total de ciclos.

$$\frac{\text{Total Cycles - Stall Cycles}}{\text{Total Cycles}}$$